

คู่มือการเขียนโปรแกรม
การควบคุมมือกลโดยใช้หลักการตรวจสอบการเคลื่อนไหวของมือ

CONTROLLER HAND ROBOT
BY USING HAND-MOTIVE DETECTION

โดย

นาย ประเสริฐ เดชกมล

นาย อรรคพร ผิวแดง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

สาขาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2545

บทที่ 1

โปรแกรมควบคุม 89c51

โปรแกรมควบคุม 89c51 นั้นเป็นโปรแกรมที่ทำการส่งข้อมูลที่ได้รับจากตัวเซนเซอร์ ซึ่งไอซี 89C51 จะเป็นตัวจัดการกระบวนการการเลือกเซนเซอร์ที่จะส่งไปให้ผู้ใช้งานคอมพิวเตอร์ได้ใช้งานได้

โดยการป้อนข้อมูลบันทึกลงไปไอซี 89C51 นั้น เราจะต้องทำการเขียนโปรแกรมโดยสามารถใช้โปรแกรมภาษาต่างๆ เช่น แอสเซมบลี ภาษาซี เป็นต้น โดยเราต้องมีการเลือกใช้คอมไพเลอร์ที่สนับสนุนกับการแปลงข้อมูลเพื่อจะได้ทำการบันทึกลงไอซีได้อีกด้วย

1.1 โครงสร้างการทำงานหลักๆ ของโปรแกรมที่จะทำการบันทึกลงไอซี 89C51

- ส่วนของการเลือกตัวเซนเซอร์ที่จะทำการรับเข้าตัว 89C51 ในแต่ละเวลาขณะหนึ่งๆ
- ส่วนของการส่งคำสั่งค่าสำหรับการติดต่อการส่งข้อมูลผ่านทางพอร์ตอนุกรม
- ส่วนของการส่งค่าที่ได้รับจากตัวเซนเซอร์แล้วทำการส่งบอกคอมพิวเตอร์ได้ว่าเป็นค่าจากเซนเซอร์ตัวไหน และมีค่าจากเซนเซอร์นั้นเป็นเท่าไร

จากนั้นเราก็จะทำการเลือกใช้คอมไพเลอร์ที่เหมาะสมและสะดวก ซึ่งในที่นี้โปรแกรมของเราค่อนข้างมีความเรียบง่าย เพราะว่ามีโครงสร้างของโปรแกรมที่ไม่ยุ่งยากซับซ้อนดังจะเห็นได้จากโครงสร้างตามหัวข้อ 1.1 นี้ ดังนั้น เราจึงเลือกใช้ภาษาแอสเซมบลีในการเขียน โดยคอมไพเลอร์ที่เราใช้นั้นมีชื่อว่า SXA51 ซึ่งเมื่อเราเขียนโปรแกรมในภาษาแอสเซมบลี (ซึ่งจะมีนามสกุลของไฟล์ว่า .ASM) แล้ว SXA51 จะทำการแปลงไฟล์ .ASM ที่ได้ไปเป็นนามสกุล .HEX ถ้าไม่พบข้อผิดพลาดในการเขียนโปรแกรม .ASM นี้ ซึ่งหลังจากนั้น เราก็จะนำไฟล์ .HEX นี้ไปใช้ในการบันทึกข้อมูลลงไอซี 89C51 ด้วยเครื่องบันทึกอีกทีหนึ่ง

1.2 การใช้งานโปรแกรม SXA51 ASSEMBLER

1.2.1 รายละเอียดทั่วไปของ SXA51 ASSEMBLER

SXA51 เป็นโปรแกรม 8051/51 ครอสแอสเซมเบลอร์ ใช้สำหรับทำการแปล SOURCE PROGRAM ซึ่งเขียนเป็นภาษาแอสเซมบลีของ MCS-51 (FILE.ASM) จะได้เอาท์พุทไฟล์เป็น INTEL HEX FILE (FILE.HEX) การเขียน SOURCE FILE ให้กระทำจากโปรแกรม EDITOR ใดๆ ก็ได้ จากนั้นจึงนำมาแปลด้วย SXA51 ซึ่งทำได้โดยการเรียกใช้โปรแกรม SXA51.EXE และตามด้วยชื่อ SOURCE PROGRAM ที่เขียนขึ้น การแปลของโปรแกรม SXA51 จะกระทำในแบบ 2 PASS ASSEMBLER ซึ่ง

หากมีข้อผิดพลาดใด ๆ SXA51 จะแสดงให้ทราบในขณะที่แปล ลักษณะของ SOURCE PROGRAM จะเป็นไปตามมาตรฐานของการเขียนโปรแกรม ASSEMBLY ทั่วไป

1.2.2การใช้งาน SXA51 ASSEMBLER

โปรแกรมแอสเซมเบลอร์ที่ใช้งานคือ SXA51.EXE โดย ASSEMBLER จะทำการแปล SOURCE FILE ซึ่งถูกกำหนดไว้เป็น FILE.ASM เมื่อทำการแปลแล้ว จะได้ FILE OUTPUT เป็น INTEL-HEX FILE ซึ่งถูกกำหนดไว้เป็น FILE.HEX โดย FILE.HEX นี้เอง ที่ผู้ใช้สามารถนำไปโหลดเข้า บอร์ดต่างๆ หรืออัดเข้าตัว EPROM เพื่อนำไปทดสอบโปรแกรมตามที่เขียนได้ นอกจากนี้ SXA51 ยังสามารถสร้าง LISTING FILE ต่าง ๆ ได้ตามรูปแบบของ OPTION ดังนี้

A>SXA51 <-OPTION> SOURCE FILE

โดย

SOURCE FILE : คือชื่อ FILE ของ โปรแกรม กรณีที่ไม่ใส่نامสกุล ของ FILE ตัว SXA51 จะกำหนดให้เป็น .ASM โดยอัตโนมัติ

<-OPTION> : เป็นส่วนของการกำหนด OPTION ต่างๆ ประกอบด้วยเครื่องหมาย “-“ และตามด้วยตัวอักษรย่อเพียงตัวเดียว ซึ่งการกำหนด OPTION นี้ สามารถกำหนดได้มากกว่า 1 ตัวในการแปลแต่ละครั้ง รายละเอียดของ OPTION จะมีดังนี้

-L หมายถึงให้สร้าง FILE แสดงรายละเอียด (LISTING FILE)

โดย OUTPUT FILE ที่ได้คือ FILE.LST

-N หมายถึงให้ทำการคอมไพล์ โดยไม่สร้าง FILE OUTPUT ใด ๆ ทั้งนี้ เพื่อการตรวจสอบความถูกต้องของโปรแกรมอย่างรวดเร็ว

-C หมายถึงให้สร้าง FILE แสดงรายละเอียด เช่นเดียวกับ -L โดยเพิ่ม symbol cross-reference ไว้ที่ท้ายของ FILE ทั้งนี้เพื่อความสะดวกในการค้นหาตำแหน่งต่างๆ ตามชื่อ LABEL

-D หมายถึง ให้แสดง PROCESS ขณะที่กำลังทำการ ASSEMBLER โดย โปรแกรมจะแสดงเลขที่บรรทัดที่กำลังแปลอยู่

ตัวอย่างการใช้คำสั่ง

A>SXA51 J31EX4

หมายถึง SOURCE : J31EX4.ASM

LISTING : ไม่สร้าง

HEX : สร้าง HEX FILE ชื่อ J31EX4.HEX

A>SXA51 -L J31EX4.ASM

หมายถึง SOURCE : J31EX4.ASM
LISTING : สร้าง LISTING FILE ชื่อ J31EX4.LST
HEX : สร้าง HEX FILE ชื่อ J31EX4.HEX

A>SXA51 -N J31EX4.ASM

หมายถึง SOURCE : J31EX4.ASM
LISTING : ไม่สร้าง
HEX : ไม่สร้าง

ข้อกำหนดของ SOURCE FILE

1. ในแต่ละบรรทัดต้องมีคำสั่งในภาษาแอสเซมบลี หรือ ASSEMBLY DIRECTIVE เพียงหนึ่งคำสั่งเท่านั้น
2. LABEL DECLARATION ทุก LABEL ต้องปิดท้ายด้วย “:”
3. โปรแกรม SXA51 จะมองตัวอักษรตัวเล็กหรือตัวใหญ่เหมือนกัน เช่น hh: จะเหมือนกับ HH:
4. บรรทัดว่างในโปรแกรมต้นแบบจะยังคงปรากฏอยู่ใน FILE แสดงรายละเอียด (FILE.LST) แต่ใน FILE แสดงรหัสภาษาเครื่อง (FILE.HEX) จะถูกตัดทิ้งไป
5. SOURCE FILE ในแต่ละบรรทัดต้องประกอบด้วยตัวอักษรไม่เกิน 128 ตัวอักษร ซึ่งในแต่ละบรรทัดต้องมีรูปแบบดังนี้

[LABEL:] OPERATION [OPERAND1],[OPERAND2] [;COMMENT]

LABEL	เป็นกลุ่มของตัวอักษรและตัวเลขที่จะใช้เป็นตัวแปร หรือตำแหน่งหน่วยความจำสำหรับอ้างอิงในโปรแกรม LABEL ต้องขึ้นต้นด้วยตัวอักษร A-Z หรือ “_” “.” “?” และต้องปิดท้ายด้วย “:” เสมอ
OPERATION	ส่วนนี้จะป็นคำสั่งมาตรฐานของ MCS51 เช่นคำสั่ง MOV INC DEC ADD
OPERAND	เป็นกลุ่มของตัวอักษรและตัวเลข ซึ่งอาจจะอยู่ในรูปของการกระทำทางคณิตศาสตร์ หรือการกระทำทางด้าน LOGIC
COMMENT	หมายถึงประโยคหรือข้อความที่ตามหลังเครื่องหมาย “:” โดยทั่วไปมักจะใช้อธิบายการทำงานของโปรแกรมสั้นๆ เพื่อให้ทำความเข้าใจได้ง่ายขึ้น เครื่องหมาย “:” จะอยู่ส่วนใดของโปรแกรมก็ได้ ซึ่งข้อความที่ตามหลังเครื่องหมาย “:” จะไม่มีผลต่อการแปลของ ASSEMBLER

ASSEMBLER DIRECTIVE

DB expression [,expression...]

DB (DEFINE BYTE) คำสั่งนี้ใช้กำหนดค่าข้อมูลเป็นไบต์ลงในหน่วยความจำโปรแกรม สามารถใส่ข้อความรหัสแอสกี(ASCII STRING) ลงตำแหน่งในหน่วยความจำได้ โดยข้อความนั้นต้องอยู่ในเครื่องหมาย QUOTES “”และยังกำหนดค่าข้อมูลที่เป็นตัวเลขและตัวอักษรในบรรทัดเดียวกันได้โดยการคั่นด้วยเครื่องหมาย “,” เช่น

```
POINTER DB 'This is a string',0
```

DS expression

DS (DEFINE SPACE) ใช้สำหรับกำหนดที่ว่างเป็นจำนวนไบต์สำหรับเก็บค่าตัวแปรหรือใช้เป็นส่วนของ BUFFER

DW expression [,expression...]

DW (DEFINE WORD) ใช้ในการเขียนข้อมูล 16 บิต ของหน่วยความจำโปรแกรมโดยการกำหนดค่าข้อมูลไบต์ต่าก่อนแล้วตามด้วยข้อมูลไบต์สูง โดยข้อมูลแต่ละ WORD ต้องคั่นด้วยเครื่องหมาย“,” เช่นเดียวกับคำสั่ง DB

ORG expression

ORG (ORIGIN) ใช้สำหรับกำหนด ADDRESS เริ่มต้นของโปรแกรม (ORIGIN PROGRAM LOCATION)

END

END คำสั่งนี้มีไว้เพื่อบอกให้ ASSEMBLER ทราบถึงจุดสิ้นสุดของโปรแกรมว่าอยู่ที่ตำแหน่งใดคำสั่งนี้จำเป็นต้องใส่ไว้ท้ายโปรแกรมด้วย

SYMBOL_NAME EQU expression

SYMBOL_NAME BIT expression

สำหรับการกำหนดค่าให้กับ SYMBOL_NAME (หรือที่เรียกกันว่า LABEL) โดยสามารถใช้คำสั่ง EQU หรือคำสั่ง BIT ก็ได้ ซึ่งจะทำงานเหมือนกันทุกประการ

SYMBOL_NAME SET expression

คำสั่ง SET จะทำงานเหมือนกับ EQU ทุกประการ แต่จะสามารถกำหนดให้ SYMBOL_NAME มีค่าได้หลาย ๆ จุดตามที่ต้องการ

Expression

การกำหนดค่าใน Expression ของแต่ละคำสั่งในภาษา ASSEMBLER นั้น สามารถใส่ตัว OPERATOR ต่าง ๆ เพื่อความสะดวกในการใช้งานได้ รวมทั้งยังกำหนดเป็นเลขฐานต่างๆ ได้ด้วยสรุปได้ดังนี้

1. โดยปกติตัวเลขจะถือว่าเป็นเลขฐานสิบเสมอ แต่ถ้าต้องการกำหนดเป็นเลขฐานสิบหก(HEXADECIMAL) จะต้องเขียนนำหน้าด้วยตัวเลขและลงท้ายด้วยตัวอักษร H เช่น 12H, 0F7H และถ้าต้องการกำหนดเป็นเลขฐานแปด (OCTAL) ให้ลงท้ายด้วยตัวอักษร O หรือ Q และถ้าต้องการเป็นเลขฐานสอง(BINARY) ให้ลงท้ายด้วย B
2. สัญลักษณ์ \$ จะหมายถึงการอ้างตำแหน่ง ADDRESS ในบรรทัดนั้น ๆ
 1. สามารถใช้คำว่า HIGH หรือ LOW เพื่อกำหนดค่าเฉพาะส่วน สูง หรือ ต่ำได้ เช่นถ้ามีการกำหนดค่าให้ชื่อ TEST EQU 1234H สามารถเรียกค่าส่วนสูงได้คือ MOV A,#HIGH TEST ซึ่งจะทำให้ได้ค่า 12H ตามต้องการ
 2. สามารถกำหนด OPERATOR ในการคำนวณได้คือ + - * / ^ (exponentiation)
 3. สามารถกระทำทางด้าน LOGIC ได้คือ MOD SHR SHL AND OR XOR

BIT OPERATION

SXA51 มีความสามารถอ้างตัวแปรในระดับ BIT ได้ โดยใช้หลักการ h.m ซึ่ง n คือค่า BYTE ADDRESS และ m คือ หมายเลข BIT ADDRESS เท่านั้น เช่น 22H.3 จะมีค่าเท่ากับ 13H รวมทั้งจะใช้กับชื่อมาตรฐานได้ด้วย เช่น ACC.7 P1.2

SXA51 ERROR MESSAGE

Byte

- | | |
|---|---|
| B | Value out-of-bounds Must be between 0 and 255 |
|---|---|

Control

- | | |
|----|---|
| C1 | A primary control must be used before any code generation |
| C2 | Unrecognized control |

Expression

- | | |
|----|--------------------------------|
| E1 | Unbalanced parentheses |
| E2 | cannot translate variable name |
| E2 | Divide by zero |
| E4 | Incorrect bit address |
| E5 | General expression error |

- E6 Radix error
- E7 Bit.n: n must be between 0-7

Include

- I Include nesting too deep

Multiply define

- M1 Label multiply defined
- M2 EQU or SET multiply defined

Offset error

- 01 ACALL or AMP must be inside 4096 byte syntax
- 02 SJMP must be within +127 or -128

Syntax

- S General
- S1 First char. on line must be alphabetic
- S2 Label error: probably a reserved word
- S3 First char.(after label) must be alphabetic
- S4 Reserved word in EQU or SET
- S5 Incorrect use of opcode
- S6 Cannot evaluate ORG expression
- S7 Incorrect register or data type

- S8 Cannot DEC DPTR
- S9 Expected 'A'
- S10 Expected comma
- S11 Expected 'AB'
- S12 Expected '@A+DPTR'
- S13 Expected '@A+PC' or '@A+DPTR'
- S14 Expected 'A,@DPTR','A,@R0' or 'A,@R1'
- S15 Expected '@R0' or '@R1'

- S16 SETB operand must be Bit or C
- S17 String must be enclosed in quotes or is too long

Unrecognized

- U General
- U0 Opcode

ข้อเสนอแนะในการใช้ SXA51

1. BUG ของ SXA51 ที่ต้องหลีกเลี่ยง

ในกรณีที่ผู้ใช้ใช้คำสั่ง MOV BUFFER,#35H โดยชื่อ BUFFER คือส่วนหนึ่งในโปรแกรมที่ได้ทำการกำหนดเอาไว้ด้วยคำสั่งเทียมน DS หรือ คำสั่งเทียมน EQU ในกรณีนี้ถ้าการกำหนดชื่ออยู่ก่อนการใช้อ้างอิงก็จะมีปัญหาอะไร แต่ถ้าอยู่หลังการใช้อ้างอิงจะเกิดปัญหาการแปลไม่ถูกต้อง และที่สำคัญคือ ไม่มีการบอก ERROR ให้ผู้ใช้ได้ทราบ ดังนั้นขอให้จำหลักการง่าย ๆ ดังนี้คือ การกำหนดค่าให้กับ LABEL ไม่ว่ามาจากคำสั่งเทียมน DS หรือ EQU ก็ตาม ให้กระทำที่ต้นโปรแกรมเสมอ ซึ่งก็จะใช้งานไปได้อย่างดี

2. เทคนิคการใช้ตัวแปร \$

คือตัวแปรที่ใช้แทนค่า ADDRESS ของบรรทัดนั้น ๆ ซึ่งจะทำให้ผู้ใช้สามารถอ้างอิงถึง ADDRESS ในระยะใกล้ ๆ ได้ โดยที่ไม่ต้องกำหนด LABEL ให้มากเกินไปจนจำเป็น

ตัวอย่าง CJNE A,#1,\$+6 ;ถ้า A ไม่เท่ากับ 1 ให้ JUMP ไปอีก 6 BYTE คือไปทำคำสั่ง CJNE
LJMP MAIN1 ต่อไป นั่นเอง

CJNE A,#2,\$+6

LJMP MAIN2

CJNE A,#3,\$+6

LJMP MAIN3

LJMP MAIN4

3. ความสามารถในการอ้าง LABEL เฉพาะส่วน

กรณีที่ผู้ใช้กำหนด LABEL ดังนี้ TEST EQU 147FH ชื่อ TEST จะมีค่าตัวแปรเป็น 16 BYTE และถ้าผู้ใช้ต้องการอ้างอิงแต่ละ BYTE ของชื่อนี้ก็สามารถทำได้ดังนี้

MOV R2,#HIGH TEST ;จะได้ค่าเท่ากับ 14H

MOV R3,#LOW TEST ;จะได้ค่าเท่ากับ 7FH

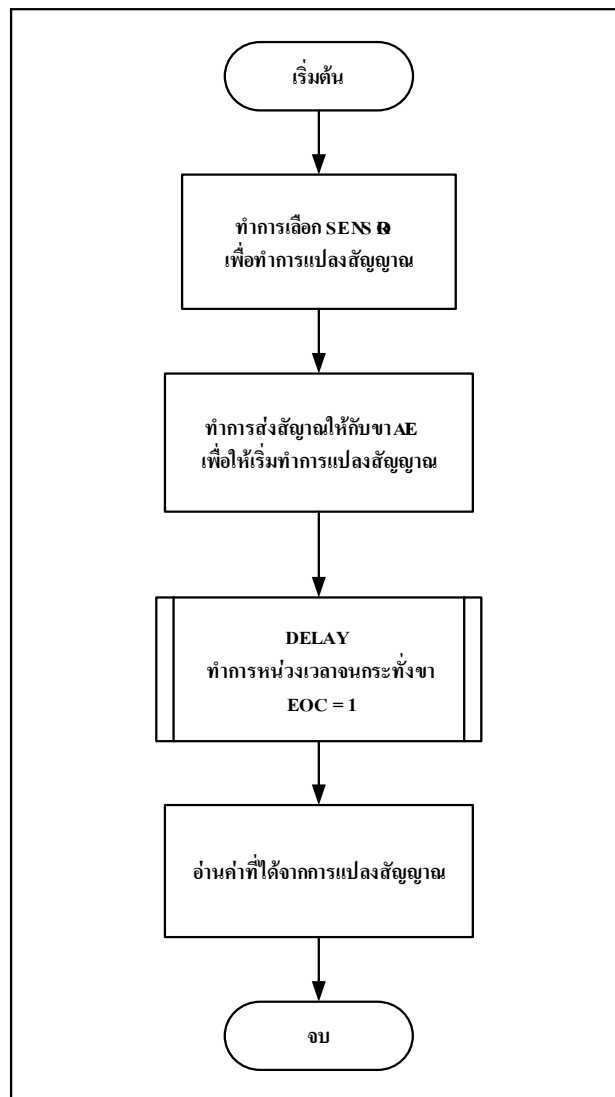
บทที่ 2

การเขียนโปรแกรมบนไอซี 89C51

2.1 โปรแกรมการอ่านค่าจากไอซีแปลงสัญญาณจากอนาลอกเป็นดิจิตอล

2.1.1 แนวคิดในการเขียนโปรแกรม

ในการเขียนโปรแกรมการอ่านค่าจากตัวแปลงสัญญาณนั้นเราจะต้องทำการส่งสัญญาณเริ่มต้นพร้อมทั้งแอดเดรส ของเซนเซอร์ที่ต้องการทำการตรวจวัดเรา เพื่อให้เวลาในการแปลงสัญญาณของไอซีแปลงสัญญาณจากอนาลอกเป็นดิจิตอลเบอร์ ADC0817 แล้วเมื่อมีการแปลงข้อมูลเสร็จ ADC0817 จะส่งสัญญาณออกมาทางขา EOC เมื่อใดที่สัญญาณ EOC เป็นหนึ่งเราก็ทำการอ่านข้อมูลที่ได้จาก ADC0817 ได้ โดยในที่นี้การรอจะใช้การหน่วงเวลา



รูปที่ 2.1 รูปแสดงการออกแบบโปรแกรมการอ่านค่า

2.1.2 การเขียนโปรแกรมอ่านค่าจากไอซี แปลงสัญญาณจากอนาลอกเป็นดิจิตอล

ในที่นี้เราจะยกตัวอย่างการอ่านค่าจากไอซี ADC0817 มาเพื่อให้เข้าใจการทำงานของ ไอซี ADC0817 ก่อนอื่นเราต้องสร้างฟังก์ชันในการหน่วงเวลาเพื่อรอการแปลงข้อมูล (ให้ชื่อฟังก์ชัน Delay)

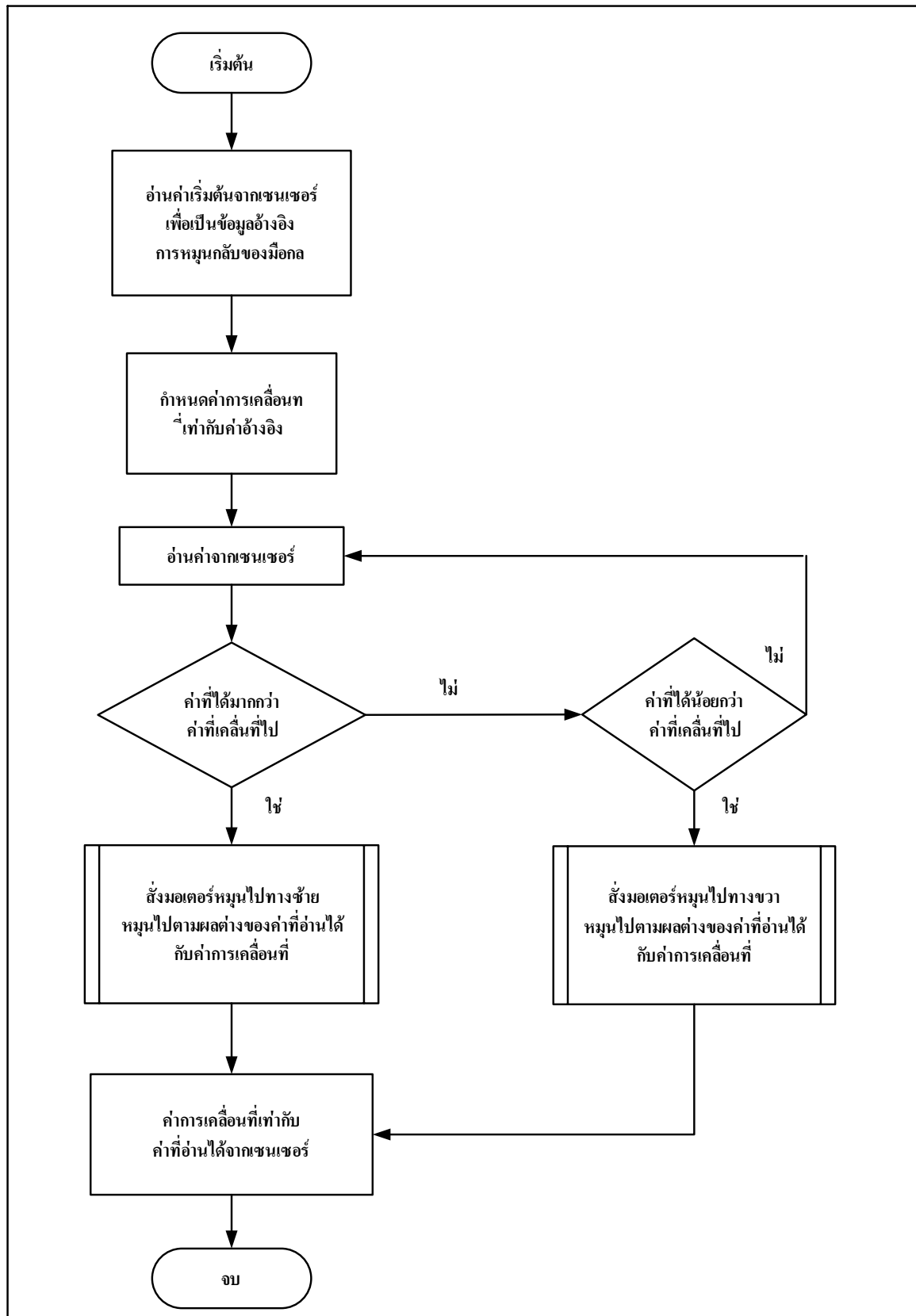
ดังนั้นเราทำการส่งสัญญาณไปยัง ขา ALE ของ ADC0817 แล้วทำการรอการแปลงสัญญาณแล้วกลับมาตรวจสอบสัญญาณจากขา EOC ของ ADC0817 ว่าเป็น 1 หรือยัง เมื่อเป็น 1 แล้วก็ทำการอ่านค่าจากพอร์ทดังตัวอย่างต่อไปนี้

```
MOV P2,#00000000B ;ส่งข้อมูลแอดเรส 000 (ในที่นี้ใช้บิต 0-3 เป็นแอดเดรส)
SETB P2.4
CLR P2.4 ;ส่งสัญญาณให้กับขา ALE
CALL DELAY ;ทำการรอจน ขา EOC เป็น 1
MOV A,P1 ;อ่านค่าจากพอร์ท ก็จะได้อ่านค่าที่ได้มาจากการแปลงสัญญาณ
```

2.2 โปรแกรมทำการตรวจสอบค่า

2.2.1 แนวคิดในการตรวจสอบค่า

เมื่อเราได้ทำการอ่านค่าที่ได้จากการแปลงค่าเราก็ทำการนำค่าที่ได้นั้นมาทำการตรวจสอบเพื่อที่จะทำการสั่งให้มอเตอร์ทำการขับเคลื่อน โดยการขับเคลื่อนของมอเตอร์ว่าจะต้องการเคลื่อนที่ไปที่สแต๊ป ดังนั้นเราจะต้องมีการอ่านค่าเริ่มต้นเพื่อเก็บค่าที่ได้มาทำเป็นตัวกำหนดการหมุนกลับของมอเตอร์เนื่องจากการออกแบบมือกลเพื่อให้สัมพันธ์กับการเคลื่อนที่ของมือที่ทำการควบคุมจะต้องมีการอ่านค่าเริ่มต้น ทั้งนี้ค่าเริ่มต้นนั้นจะต้อง วางมือและมือกลในรูปแบบเดียวกันเพื่อให้ได้การเคลื่อนไหวกวที่เหมือนกัน ดังแสดงในรูป 2.2



รูปที่ 2.2 รูปแสดงแนวคิดการออกแบบโปรแกรมการตรวจสอบค่า

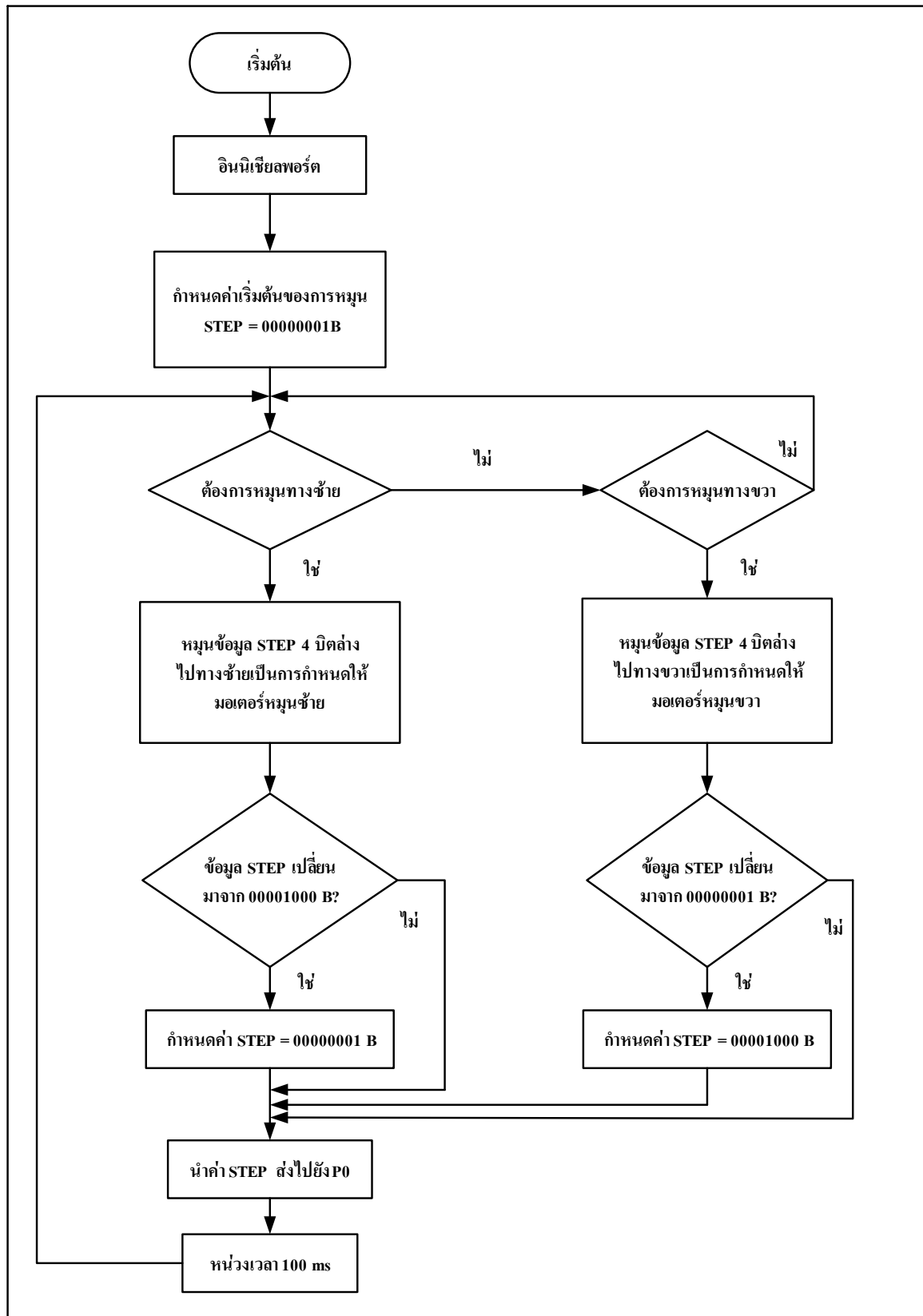
2.2.2 การเขียนโปรแกรม

เริ่มต้นจากการอ่านค่าที่ได้จากเซนเซอร์ทุกตัวเพื่อเก็บค่าเริ่มต้น เพื่อใช้เป็นลิมิตของการเคลื่อนที่ เวลาเมื่อกลเคลื่อนที่กลับ และส่วนของการเคลื่อนที่นั้นก็ทำการอ่านค่าเข้ามาได้ก็ทำการเปรียบเทียบกับค่า ก่อนหน้าว่ามีการเปลี่ยนแปลงอย่างไร ก็ทำการสั่งให้มอเตอร์เคลื่อนที่หมุนตามเข็ม หรือทวนเข็มนาฬิกา ตามค่าที่เปรียบเทียบ และจะหมุนไปมากน้อยก็อยู่กับค่าที่เปรียบเทียบได้

2.3 โปรแกรมการขับเคลื่อนมอเตอร์

2.3.1 แนวคิดในการขับเคลื่อนมอเตอร์

ในวงจรได้มีการขยายพอร์ตโดยใช้ 8255 เพื่อให้มีการทำงานของ สเต็ปเปอร์มอเตอร์ได้หลายตัว ดังนั้นการควบคุมจะต้องทำการควบคุมการทำงานผ่าน 8255 ทั้งนี้การสั่งมอเตอร์ผ่าน 8255 หรือการสั่ง หมุนมอเตอร์โดยใช้ไมโครคอนโทรลเลอร์โดยตรงก็ตามจะมีวิธีการเหมือนกันคือทำการ วนลูปในการขับเคลื่อนโดยการเลื่อนบิตเพื่อให้สเต็ปเปอร์มอเตอร์ทำงาน แต่ในส่วนของการควบคุมผ่าน 8255 จะต้องทำการเลือกพอร์ตก่อน ดังแสดงในรูป 2.3



รูปที่ 2.3 รูปแสดงการออกแบบโปรแกรมการขับมอเตอร์

2.3.2 การเขียนโปรแกรมการขับมอเตอร์

เพื่อให้ง่ายต่อความเข้าใจจะแสดงให้เห็นในลักษณะของการขับมอเตอร์ 89C51 พอร์ต 0 ดังนั้นจึงขอยกตัวอย่างการสั่งขับเคลื่อนมอเตอร์ผ่านพอร์ต 0 ของไมโครคอนโทรลเลอร์

```
STEP      EQU  030H
          ORG  0000H
          MOV  P0,#00H
MAIN:      MOV  STEP,#00000001B
          MOV  P0,STEP

LOOP:      CJNE A,#01H,ROTATE_LEFT
          AJMP ROTATE_RIGHT
```

;เป็นการหมุนมอเตอร์ทางด้านซ้าย

```
ROTATE_LEFT: MOV  A,STEP
            SWAP  A
            CLR   C
            RLC   A
            JNC   NEXT_LEFT
            MOV   A,#00010000B
NEXT_LEFT:  SWAP  A
            MOV   STEP,A
            MOV   P0,STEP
            ACALL DELAY_100ms
            AJMP  LOOP
```

;เป็นการหมุนมอเตอร์ทางด้านขวา

```
ROTATE_RIGHT: MOV  A,STEP
            CLR   C
            RRC   A
            JNC   NEXT_RIGHT
            MOV   A,#00001000B
NEXT_RIGHT:  MOV   STEP,A
            MOV   P0,STEP
```

หมายเหตุ การอ้างพอร์ตการทำงานต่างๆ ควรจะต้องให้ถูกต้องและตรงกับวงจรที่ได้ออกแบบไว้เพื่อให้ได้ค่าออกมาที่ถูกต้องและสามารถทำงานได้